Scaling Laws for Language Models

From Power Laws to Compute-Optimal Training

CS115B – Spring 2026

James Pustejovsky

April 13, 2026

Roadmap (90 Minutes)

Part I – Foundations: Why Scale Matters ∼20 min

- The empirical surprise: performance improves predictably with scale
- Measuring scale: parameters (N), data (D), compute (C)
- Cross-entropy loss as the universal yardstick

Part II – Kaplan et al. (2020): The First Scaling Laws ∼25 min

- Power-law relationships across seven orders of magnitude
- Optimal compute allocation and early stopping

Part III – Hoffmann et al. (2022): Chinchilla ∼25 min

- Three approaches to finding the optimal frontier
- "Scale parameters and data equally"
- Chinchilla vs. Gopher: the empirical validation

Part IV – Beyond Chinchilla: Open Questions ∼20 min

- Inference-time compute, over-training, and data quality
- Do scaling laws still hold? What breaks?

The Empirical Observation

The Bitter Lesson (Sutton, 2019)

General methods that leverage computation are ultimately the most effective.

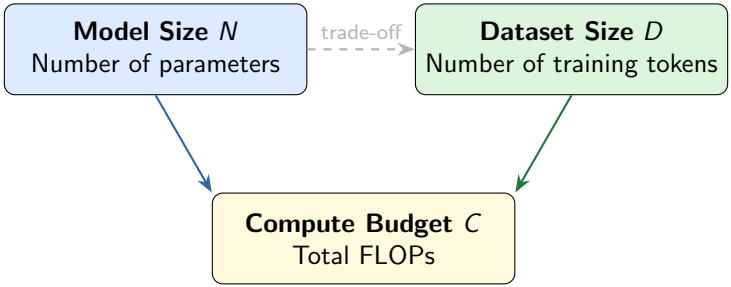
The history of deep learning in NLP:

- 2013 – Word2Vec: 300-dim vectors, $\sim$1M vocabulary
- 2018 – BERT-Large: 340M parameters
- 2019 – GPT-2: 1.5B parameters
- 2020 – GPT-3: 175B parameters
- 2022 – PaLM: 540B parameters
- 2023 – Llama 2: 7B–70B parameters
- 2024–25 – Llama 3, Gemini, Claude: trillions of tokens

Key question: Is this growth principled, or are we just throwing compute at the wall?

Three Axes of Scale

Every training run is defined by three quantities:



$C \approx 6ND$ *(Kaplan et al., 2020)*

Given a fixed compute budget C, how should we allocate between N and D?

This is the **central question** of scaling law research.

Computing FLOPs

For a standard Transformer with N parameters, trained on D tokens:

Forward pass per token $\approx 2N$ multiply-accumulate operations.
Backward pass $\approx 2\times$ forward pass.

The $6ND$ Approximation

$$C \approx 6\,N\,D \quad \text{(total training FLOPs)}$$

Breakdown for a single layer (forward only):

- Attention QKV projections: $2 \times 3 \times T \times d_{\text{model}} \times d_{\text{model}}$
- Attention logits: $2 \times T^2 \times d_{\text{model}}$
- FFN (two linear layers): $2 \times T \times (d_{\text{model}} \times d_{\text{ff}} + d_{\text{ff}} \times d_{\text{model}})$

With $d_{\text{ff}} = 4\,d_{\text{model}}$ and L layers: forward FLOPs $\approx 2N \times T$ (embedding terms small for large N).

Cross-Entropy Loss as the Yardstick

Why cross-entropy loss?

- Smooth, continuous, differentiable – ideal for curve fitting
- Directly connected to perplexity: $\text{PPL} = e^L$
- Well-defined across model sizes and architectures
- Correlated with downstream task performance (generally)

The loss function:

$$L = -\frac{1}{D}\sum_{i=1}^{D} \log P_\theta(x_i \mid x_{<i})$$

Lower bound: The entropy of natural text ≈ 1.69 nats (Hoffmann et al., 2022).

Important: Scaling laws describe how L varies with N, D, and C – *not* downstream task accuracy directly.

Perplexity vs. Cross-Entropy

Recall the relationship:

$$\text{Perplexity} = e^{L} = e^{-\frac{1}{D}\sum_i \log P_\theta(x_i|x_{<i})}$$

Cross-Entropy L	**Perplexity**	**Interpretation**
4.0	54.6	Random-ish predictions
3.0	20.1	Small LM territory
2.5	12.2	Decent language model
2.0	7.4	Strong LM (GPT-2 level)
1.69	5.4	Entropy of English (floor)

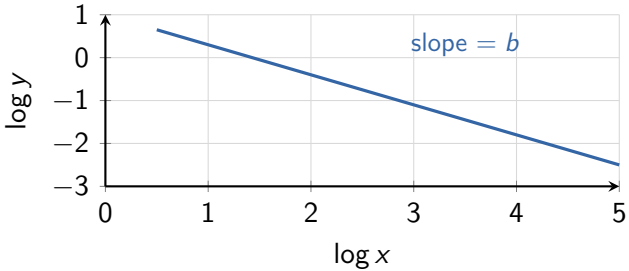
Scaling laws are typically plotted in **log-log space**: if $L \propto N^{-\alpha}$, then $\log L$ vs. $\log N$ is a straight line with slope $-\alpha$.

What Is a Power Law?

A **power law** is a relationship of the form:

$$y = a \cdot x^b$$

In log-log space: $\log y = \log a + b \cdot \log x$ (a straight line!).



Power laws are ubiquitous in nature: city sizes, earthquake magnitudes, word frequencies (Zipf's law!). The discovery that LM loss follows power laws was a major empirical finding.

From Zipf to Scaling Laws

NLP has long known about power laws:

Zipf's Law: The frequency of a word is inversely proportional to its rank:

$$f(r) \propto r^{-\alpha}, \quad \alpha \approx 1$$

Heaps' Law: Vocabulary size grows sub-linearly with corpus size:

$$|V| \propto D^{\beta}, \quad \beta \approx 0.5$$

Scaling laws for LMs (Kaplan et al., 2020) are a *new* power law:

$$L(N) \propto N^{-\alpha_N}, \qquad L(D) \propto D^{-\alpha_D}, \qquad L(C) \propto C^{-\alpha_C}$$

The surprise: These hold over *seven orders of magnitude* – from 768-parameter models to 1.5B-parameter models – with remarkable predictive accuracy.

Kaplan et al. (2020): Overview

Paper

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, Dario Amodei.
Scaling Laws for Neural Language Models. arXiv:2001.08361.

Key contributions:

1. Empirical power-law relationships: L vs. N, D, and C
2. Architecture details (width, depth, #heads) matter *much less* than scale
3. Larger models are more **sample-efficient**
4. Optimal training: train *very large* models on *relatively little data*, stop early

Training setup: Decoder-only Transformers on WebText2, varying N from $\sim$768 to $\sim$1.5 B.

The Core Power Laws

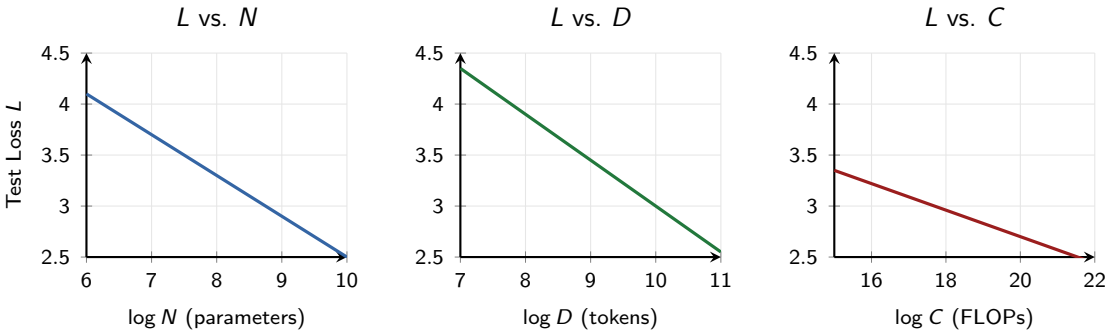
Kaplan et al. found three independent power-law relationships:

Relationship	**Formula**	**Exponent**
Loss vs. Parameters	$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$	$\alpha_N \approx 0.076$
Loss vs. Data	$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}$	$\alpha_D \approx 0.095$
Loss vs. Compute	$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$	$\alpha_C \approx 0.050$

Each law holds when the *other* factors are not bottlenecks (i.e., sufficient data when varying N, sufficient model size when varying D).

Visualizing the Power Laws

In log-log space, these become straight lines:



Schematic illustration. The actual plots from Kaplan et al. span 7+ orders of magnitude on the x-axis, with thousands of data points falling tightly on the line.

Architecture Doesn't Matter (Much)

Surprising finding: Once N is fixed, architectural details have minimal effect on L:

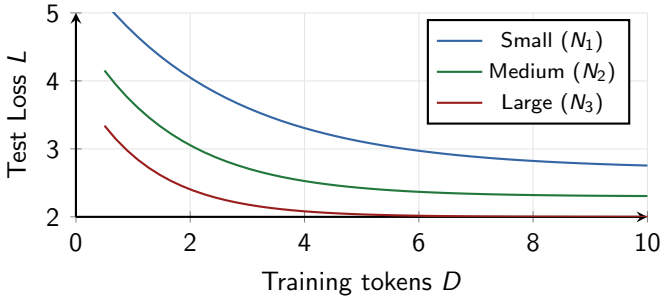
Varied	**Result**
Width vs. depth	Minimal effect across wide range of d_{model}/L ratios
Number of heads	No significant effect (given fixed d_{model})
Feed-forward ratio	$d_{\text{ff}}/d_{\text{model}} \in [1, 8]$: small effect
Context length	Minimal effect on loss (though affects downstream tasks)

Implication: The *total number of parameters* is what matters, not how they are arranged.

Caveat: This was tested within a specific family of Transformer architectures. Different activation functions, normalization schemes, or entirely different architectures might differ.

Sample Efficiency

Key finding: Larger models reach the same loss with *fewer* data points.



At the same data budget, the larger model always achieves lower loss. This suggests: if compute is limited, prefer a **larger model trained for fewer steps** over a smaller model trained longer.

Optimal Compute Allocation (Kaplan)

Given a fixed compute budget C, Kaplan et al. recommended:

Kaplan Scaling Recipe

$$N_{\text{opt}} \propto C^{0.73}, \qquad D_{\text{opt}} \propto C^{0.27}$$

Interpretation: For a $10\times$ increase in compute:

- Increase model size by $\sim 5.5\times$
- Increase training data by only $\sim 1.8\times$

This led to a clear industry trend:

- GPT-3 (175B) trained on 300B tokens
- Gopher (280B) trained on 300B tokens
- Megatron-Turing NLG (530B) trained on 270B tokens

Same data, bigger models. **Was this the right strategy?**

Kaplan's Overfitting Analysis

Kaplan et al. also derived a simple overfitting law:

$$L(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

Overfitting becomes significant when $D \lesssim 5000 \cdot N^{0.74}$.

Parameters N	**Min. tokens** D (no overfit)	**Ratio** D/N
100M	$\sim$2B	20$\times$
1B	$\sim$10B	10$\times$
10B	$\sim$55B	5.5$\times$
100B	$\sim$300B	3$\times$

Note: As models get larger, the required D/N ratio actually *decreases* – large models can "afford" to see proportionally less data. (But Chinchilla will challenge this intuition.)

Limitations of Kaplan et al.

Several methodological choices would later prove consequential:

1. **Fixed training schedule.** All models used the same cosine schedule over 130B tokens – shorter runs gave *intermediate* losses, not final losses after proper LR decay.
2. **Mostly small models.** The majority of runs used models with $<$100M parameters. Extrapolating to 175B+ introduces risk.
3. **No learning rate tuning per run.** The LR schedule was not matched to the training horizon, causing systematically higher losses for shorter training runs.
4. **Single dataset.** All experiments used WebText2.

Key consequence: Using intermediate losses from a long cosine schedule *overestimates* the loss of a model that would be trained with a properly matched shorter schedule.

This makes short training runs look worse than they are, biasing toward *larger models with less data*.

Summary: Kaplan et al. Takeaways

1. Loss follows power laws in N, D, C

2. Architecture details matter little

3. Larger models are more sample-efficient

4. Scale N faster than D: $N_{\text{opt}} \propto C^{0.73}$

5. Train large, stop early

But points 4 and 5 would be revised by Hoffmann et al. (2022), who showed that data should be scaled equally with parameters.

Hoffmann et al. (2022): Overview

Paper

Jordan Hoffmann*, Sebastian Borgeaud*, Arthur Mensch*, Elena Buchatskaya, . . . , Laurent Sifre*.
Training Compute-Optimal Large Language Models. arXiv:2203.15556.

Central claim: Current LLMs are **significantly undertrained**.

For the same compute budget, a *smaller* model trained on *more data* outperforms a larger, data-starved model.

Evidence:

- Trained 400+ models from 70M to 16B parameters, on 5B to 500B tokens
- Three independent approaches all converge on the same conclusion
- Validated with **Chinchilla** (70B params, 1.4T tokens) vs. Gopher (280B params, 300B tokens) – same compute budget

The State of LLMs in Early 2022

Model	Parameters	Training Tokens
LaMDA (Google, 2022)	137B	168B
GPT-3 (OpenAI, 2020)	175B	300B
Jurassic-1 (AI21, 2021)	178B	300B
Gopher (DeepMind, 2021)	280B	300B
Megatron-Turing NLG (Microsoft/NVIDIA, 2022)	530B	270B
Chinchilla (DeepMind, 2022)	**70B**	**1.4T**

Observation: Nearly all models trained on $\sim$300B tokens, regardless of model size. This is Kaplan's recipe in action.

Chinchilla bucks the trend: $4\times$ fewer parameters, $4\times$ more data, same total compute as Gopher.

The Central Question

Optimization Problem

$$N_{\text{opt}}(C),\ D_{\text{opt}}(C) \ = \ \arg\min_{N,D} \ L(N, D) \quad \text{s.t.} \quad \text{FLOPs}(N, D) = C$$

Given a budget of C FLOPs, what is the best (N, D) pair?

Hoffmann et al. used three independent approaches:

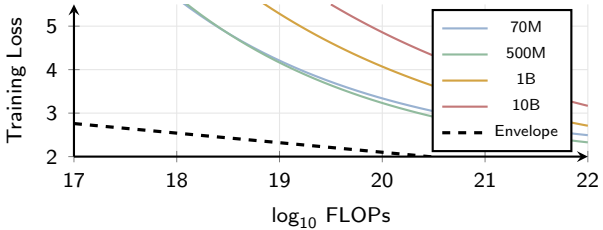
1. Approach 1: Fix model sizes, vary training tokens. Extract the *envelope* of minimum loss per FLOP.
2. Approach 2: Fix FLOP budgets (IsoFLOP profiles), vary model size. Find the model size that minimizes loss for each budget.
3. Approach 3: Fit a *parametric loss function* $\hat{L}(N, D)$ to all runs simultaneously.

All three approaches converge on the same answer.

Approach 1: Training Curve Envelope

Method:

1. Train models from 70M to 10B parameters
2. For each model size, train for 4 different cosine schedule lengths (varying by $16\times$)
3. Interpolate loss curves; for each FLOP value, find which (N, D) achieves lowest loss



The **envelope** (dashed) traces the minimum achievable loss at each FLOP budget.

Approach 1: Results

Fitting power laws to the envelope:

Approach 1 Scaling Exponents

$$N_{\text{opt}} \propto C^{0.50}, \qquad D_{\text{opt}} \propto C^{0.50}$$

Equal scaling! For every doubling of compute:

- Double the model size
- Double the training data

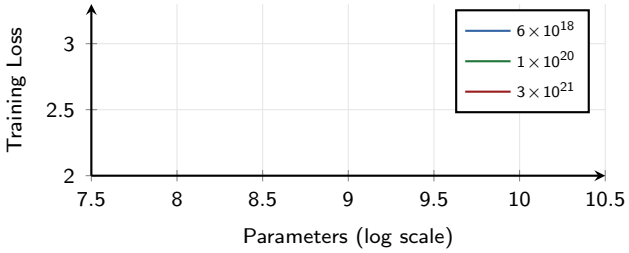
Compare with Kaplan: $N_{\text{opt}} \propto C^{0.73}$, $D_{\text{opt}} \propto C^{0.27}$.

Why the difference?

- Kaplan used intermediate losses from a fixed long cosine schedule
- Hoffmann matched the LR schedule to each training horizon
- Matching the schedule gives systematically lower final losses for shorter runs

Approach 2: IsoFLOP Profiles

Method: For each of 9 FLOP budgets (6×10^{18} to 3×10^{21}), vary model size and find the loss-minimizing N.



Each U-shaped curve has a clear **valley** – the optimal model size for that FLOP budget.
Too small $\rightarrow$ underfitting. Too large $\rightarrow$ undertrained.

Approach 2: Results

Approach 2 Scaling Exponents

$$N_{\text{opt}} \propto C^{0.49}, \qquad D_{\text{opt}} \propto C^{0.51}$$

Again: approximately **equal scaling** of parameters and data.

Advantages of IsoFLOP profiles:

- Only uses *final* losses (not intermediate training points)
- Cosine schedule properly matched to each training horizon
- Models trained up to 16B parameters (vs. 10B in Approach 1)

Projection for Gopher's budget (5.76×10^{23} FLOPs):
Optimal model: $\sim$63B parameters, trained on $\sim$1.4T tokens.

Approach 3: Parametric Loss Function

Model: Decompose the loss into three terms:

Parametric Loss Decomposition

$$\hat{L}(N, D) = \underbrace{E}_{\text{entropy of text}} + \underbrace{\frac{A}{N^{\alpha}}}_{\substack{\text{finite model} \\ \text{approximation}}} + \underbrace{\frac{B}{D^{\beta}}}_{\substack{\text{finite data} \\ \text{approximation}}}$$

Interpretation of each term:

- E – irreducible loss: the entropy of natural text (≈ 1.69 nats)
- A/N^{α} – how much worse a model of size N is vs. an ideal model
- B/D^{β} – how much worse training on D tokens is vs. infinite data

Fitted values: $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$.

Approach 3: Derivation of the Efficient Frontier

Minimize $\hat{L}(N, D)$ subject to $C = 6ND$:

$$N_{\text{opt}}(C) = G\left(\frac{C}{6}\right)^{a}, \qquad D_{\text{opt}}(C) = G^{-1}\left(\frac{C}{6}\right)^{b}$$

where:

$$G = \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}}, \qquad a = \frac{\beta}{\alpha+\beta}, \qquad b = \frac{\alpha}{\alpha+\beta}$$

Approach 3 Scaling Exponents

$$a = \frac{0.28}{0.34 + 0.28} = 0.46, \qquad b = \frac{0.34}{0.34 + 0.28} = 0.54$$

Slightly favoring data over parameters, but still approximately equal. Note the closed-form solution – quite elegant.

Comparing All Three Approaches

Approach	a **where** $N_{\text{opt}} \propto C^a$	b **where** $D_{\text{opt}} \propto C^b$
1. Training curve envelope	0.50	0.50
2. IsoFLOP profiles	0.49	0.51
3. Parametric loss function	0.46	0.54
Kaplan et al. (2020)	0.73	0.27

Consensus: $a \approx b \approx 0.5$. **Scale parameters and data equally.**

Kaplan's $a = 0.73$ now looks like an artifact of the fixed LR schedule methodology.

Practical implication: For a $10\times$ increase in compute:

- Kaplan: $5.5\times$ bigger model, $1.8\times$ more data
- Chinchilla: $\sim 3.2\times$ bigger model, $\sim 3.2\times$ more data

Compute-Optimal Model Sizes

What does the Chinchilla frontier predict?

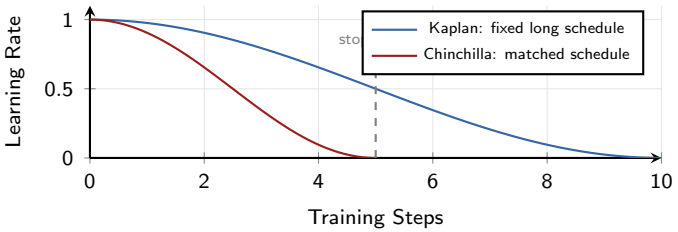
Parameters	FLOPs	Optimal Tokens	Tokens/Params
400M	1.9×10^{19}	8.0B	20×
1B	1.2×10^{20}	20.2B	20×
10B	1.2×10^{22}	205B	20×
67B	5.8×10^{23}	1.5T	22×
175B	3.9×10^{24}	3.7T	21×
280B	9.9×10^{24}	5.9T	21×
1T	1.3×10^{26}	21.2T	21×

Rule of thumb: Train on roughly 20× as many tokens as you have parameters.

A 175B model needs ∼3.7T tokens to be compute-optimal – over 10× more than GPT-3 used!

The LR Schedule Matters

Critical methodological difference between Kaplan and Hoffmann:



Kaplan's approach: Train all models with a long cosine schedule; read off intermediate loss values for shorter "training horizons."

Problem: At the intermediate read-off point, the LR hasn't decayed – the loss is artificially high. This makes short training runs look *worse* than they actually are.

Hoffmann et al. showed that overestimating the cosine cycle by >25% leads to clear performance degradation.

Chinchilla vs. Gopher: The Showdown

	Gopher	**Chinchilla**
Parameters	280B	70B
Training tokens	300B	1.4T
Training FLOPs	5.76×10^{23}	5.76×10^{23}
Layers	80	80
d_{model}	16,384	8,192
Heads	128	64
Optimizer	Adam	AdamW

Same compute budget. Same number of layers. Chinchilla is narrower (half d_{model}) but trains on 4.7$\times$ more tokens.

If Kaplan is right, Gopher should win. If Hoffmann is right, Chinchilla should win.

Chinchilla Results: Language Modeling

Chinchilla outperforms Gopher on every subset of The Pile:

Subset	Chinchilla (70B)	Gopher (280B)	Δ
Pile-CC	0.667	0.691	-0.024
PubMed Abstracts	0.559	0.578	-0.019
StackExchange	0.614	0.641	-0.027
GitHub	0.337	0.377	-0.040
ArXiv	0.627	0.662	-0.035
FreeLaw	0.476	0.513	-0.037
Books3	0.675	0.712	-0.037

Values are bits-per-byte (bpb); lower is better. Chinchilla wins uniformly despite having **4× fewer parameters**.

Chinchilla Results: Downstream Tasks

Task	Chinchilla	Gopher	GPT-3	MT-NLG 530B
Common Sense				
HellaSwag	**80.8%**	79.2%	78.9%	80.2%
Winogrande	**74.9%**	70.1%	70.2%	73.0%
BoolQ	**83.7%**	79.3%	60.5%	78.2%
Reading Comprehension				
LAMBADA	**77.4%**	74.5%	76.2%	76.6%
RACE-h	**82.3%**	71.6%	46.8%	47.9%
Knowledge				
MMLU (5-shot)	**67.6%**	60.0%	43.9%	–

Chinchilla (70B) outperforms *all* larger models on nearly every benchmark. MMLU: 67.6% exceeds human forecasters' June 2023 prediction of 63.4%.

Why Chinchilla Also Wins at Deployment

Being 4× smaller has practical advantages:

	Gopher (280B)	**Chinchilla (70B)**
GPU memory (fp16)	∼560 GB	∼140 GB
Inference cost	1×	∼ 0.25×
Fine-tuning cost	1×	∼ 0.25×
Min. GPU cluster	≥16 A100-80GB	≥4 A100-80GB

The total cost of a model = training + inference.

Training cost is paid once; inference cost is paid *every query*. A model that is deployed millions of times amortizes training cost quickly – the smaller inference footprint of Chinchilla is a compounding advantage.

The Parametric Loss Decomposition: Deeper Dive

Recall: $\hat{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$

This can be derived from a classical statistical learning decomposition:

$$L(N, D) = \underbrace{L(f^*)}_{\text{Bayes risk}} + \underbrace{L(f_N) - L(f^*)}_{\text{approximation error}} + \underbrace{L(\bar{f}_{N,D}) - L(f_N)}_{\text{estimation error}}$$

- f^* – the Bayes-optimal predictor (entropy of text)
- f_N – the best N-parameter transformer on infinite data
- $\bar{f}_{N,D}$ – the transformer after one epoch on D tokens

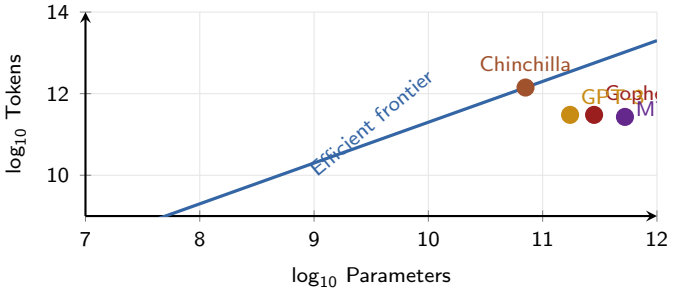
Expected scaling:

- Approximation error: $\sim N^{-1/2}$ for two-layer networks (Siegel & Xu, 2020)
- Estimation error: $\sim D^{-1/2}$ from SGD convergence rates (Robbins & Monro, 1951)

Empirically: $\alpha = 0.34$, $\beta = 0.28$ – both below $1/2$, indicating room for improvement.

IsoLoss Contours

The fitted $\hat{L}(N, D)$ generates **contour plots** in (N, D) space:



The efficient frontier passes through Chinchilla. All other models sit *below* it (too many parameters, too few tokens). Points on the frontier achieve the lowest loss for their compute budget.

Dataset Consistency

Hoffmann et al. replicated the IsoFLOP analysis on additional datasets:

Dataset	a ($N_{\text{opt}} \propto C^a$)	b ($D_{\text{opt}} \propto C^b$)
MassiveText (primary)	0.49	0.51
C4	0.50	0.50
GitHub code	0.53	0.47
Kaplan (WebText2)	0.73	0.27

The equal-scaling result is **robust across datasets**, as long as training does not exceed one epoch.

This suggests the phenomenon is a property of *optimization and model capacity*, not of any particular data distribution.

Summary: Chinchilla Takeaways

1. Scale model size and training data **equally**: $N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$

2. Most 2020–2022 LLMs were **significantly undertrained**

3. Chinchilla (70B) outperforms Gopher (280B) at same compute

4. Rule of thumb: train on $\sim 20\times$ as many tokens as parameters

5. Smaller compute-optimal models are cheaper to serve

6. The LR schedule must be matched to the training horizon

Did the Industry Listen?

Post-Chinchilla models shifted toward more data:

Model	Params	Tokens	Tokens/Params
Pre-Chinchilla			
GPT-3 (2020)	175B	300B	1.7×
Gopher (2021)	280B	300B	1.1×
Post-Chinchilla			
Chinchilla (2022)	70B	1.4T	20×
Llama 1 (2023)	65B	1.4T	22×
Llama 2 (2023)	70B	2.0T	29×
Llama 3 (2024)	70B	15T	214×
Mistral (2023)	7B	>1T	>140×

Notice: Recent models train on *far more* tokens than Chinchilla-optimal. Why?

Over-Training: When Inference Cost Dominates

Chinchilla-optimal minimizes loss for a given *training* compute budget.
But in deployment, *inference* compute dominates total cost.

The inference-aware trade-off:

- A 70B model costs $\sim 10\times$ more per query than a 7B model
- If you serve billions of queries, inference cost $\gg$ training cost
- **Solution:** Train a *smaller* model for *much longer* (over-train relative to Chinchilla-optimal)

Example: Llama 3 8B trained on 15T tokens ($\sim 1875\times$ params).
At Chinchilla-optimal, an 8B model needs only $\sim$160B tokens. So Llama 3 is trained on $\sim 94\times$ the Chinchilla-optimal data.

This is intentional: The extra training compute is a one-time cost that buys a smaller, cheaper model for inference.

The Data Wall

If we need $\sim 20\times$ tokens per parameter, and models keep growing:

Model Size	Chinchilla-Optimal Data	Challenge
1B	20B tokens	Easily available
70B	1.4T tokens	High-quality web scrapes
1T	20T tokens	Approaching total web text
10T	200T tokens	Exceeds available text

Estimates of total available text:

- Common Crawl (deduplicated): $\sim$1–5T tokens
- All books ever published: $\sim$100B–500B tokens
- All code (GitHub + others): $\sim$1–2T tokens
- Total high-quality text: $\sim$10–20T tokens (generous estimate)

Implication: We may be running out of training data before running out of compute.

Data Quality vs. Quantity

Chinchilla's scaling laws assume uniform data quality. In practice:

- **Not all tokens are equal.** A token from a textbook is worth more than a token from a random web page.
- **Data curation matters.** Llama 1's success was partly attributed to aggressive filtering and deduplication.
- **Repeating data hurts.** Training for multiple epochs shows diminishing returns and can degrade performance.
- **Synthetic data** is increasingly used to augment training sets – but scaling laws for synthetic data are not well understood.

Open question: Can we define "effective tokens" that account for data quality? Would the scaling exponents change?

Multi-Epoch Training

What happens when you train for more than one epoch?

Key findings (Muennighoff et al., 2023; others):

- Training for up to 4 epochs shows only modest degradation
- Beyond 4 epochs, performance degrades more noticeably
- The degradation is worse for larger models
- Regularization techniques (dropout, weight decay) can help

Modified scaling law for multi-epoch training:

$$\hat{L}(N, D, R) = E + \frac{A}{N^{\alpha}} + \frac{B}{D_{\text{unique}}^{\beta}} + f(R)$$

where $R = D_{\text{total}}/D_{\text{unique}}$ is the repetition factor and $f(R)$ captures the penalty.

Chinchilla laws hold best in the single-epoch regime.

Scaling Laws for Downstream Tasks

Problem: Scaling laws predict cross-entropy loss, but users care about *task performance*.

The loss-to-task-performance relationship:

- Generally monotonic: lower loss $\rightarrow$ better task performance
- But the relationship is often *nonlinear* and task-dependent
- Some tasks show "emergent" behavior: near-random until a threshold, then rapid improvement

Wei et al. (2022) argued for "emergent abilities" – capabilities that appear only at scale.

Schaeffer et al. (2024) countered that emergence is an artifact of the metric, not the model – switching from accuracy to log-likelihood removes the apparent discontinuity.

Takeaway: Scaling laws for loss are smooth; whether task performance scales smoothly depends on how you measure it.

Scaling at Inference Time

A new dimension of scaling: **test-time compute.**

Key idea: Instead of only scaling *training*, scale the amount of compute used *at inference*.

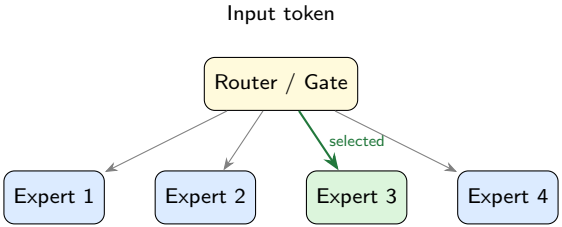
Mechanisms:

- **Chain-of-thought prompting:** Let the model "think step by step" – more output tokens = more compute per question
- **Best-of-N sampling:** Generate N answers, select the best one (e.g., using a reward model)
- **Tree search / beam search:** Explore multiple reasoning paths
- **Reasoning models** (o1, o3, R1, etc.): Trained to allocate variable compute per problem

Emerging consensus: There may be separate scaling laws for training compute vs. inference compute, and the optimal balance between them is an active research question.

Mixture of Experts and Scaling

Mixture of Experts (MoE): Not all parameters are active for every token.



Key property: Total parameters $\gg$ active parameters per token.

Scaling implications:

- MoE models get more "capacity" per FLOP
- But Clark et al. (2022) showed diminishing returns as expert count grows
- Chinchilla-style scaling laws for MoE are an active area

Examples: Switch Transformer (1.7T params), GLaM (1.2T params), Mixtral 8×7B.

What Breaks Scaling Laws?

Scaling laws are *empirical* regularities. They can break when:

1. **Data quality degrades** – adding low-quality data can increase loss despite more tokens
2. **Multi-epoch training** – repeating data violates the "infinite data" assumption
3. **Architecture changes** – scaling laws are fit to specific architectures; major innovations (e.g., SSMs like Mamba) might follow different laws
4. **Curvature at scale** – Hoffmann et al. noted slight curvature in the FLOP-loss frontier for large models, suggesting the power-law exponents may themselves change
5. **Task-specific vs. general scaling** – a model can have excellent loss but poor performance on specific tasks (alignment, safety, tool use)

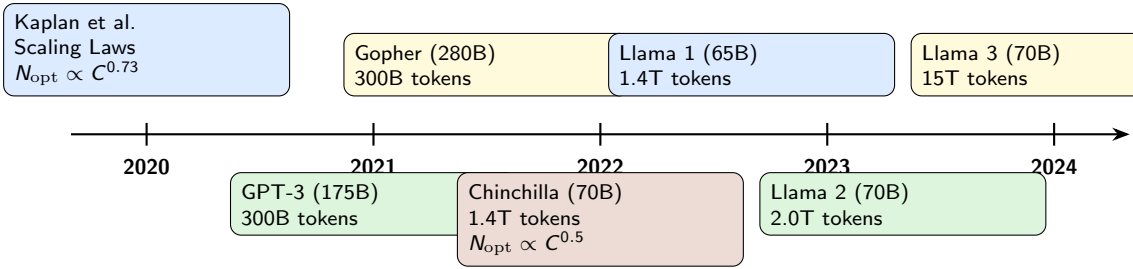
Key insight: Scaling laws are best understood as a *first-order approximation* – extraordinarily useful for planning, but not the final word.

Practical Guidelines for Researchers

If you are training a language model today:

1. **Decide your total compute budget** C first (GPUs $\times$ time $\times$ efficiency).
2. **Use Chinchilla ratios as a baseline:** $N_{\text{opt}} \approx (C/6)^{0.5}$, $D \approx 20N$.
3. **If inference cost matters** (production deployment): shift toward a smaller model trained longer ($D \gg 20N$).
4. **Match the LR schedule** to your planned training duration – do not train with a longer cosine cycle than needed.
5. **Run small-scale pilots** first: scaling laws predict that trends at small scale carry to large scale. Validate before committing to a large run.
6. **Invest in data quality.** Curated data can be worth 5–10$\times$ as much as raw web scrapes.

Timeline of Key Results



The trend is clear: **more data, not just more parameters**. The industry has moved well beyond Chinchilla-optimal into the "over-training" regime for inference efficiency.

Key Takeaways

1. **Scaling laws are power laws:** LM loss scales as $L \propto N^{-\alpha}$, $L \propto D^{-\beta}$, $L \propto C^{-\gamma}$ – predictable over orders of magnitude.
2. **Kaplan (2020):** First systematic study. Scale model size faster than data ($N \propto C^{0.73}$).
3. **Hoffmann / Chinchilla (2022):** Corrected the allocation. Scale N and D equally ($\sim C^{0.5}$ each). Most LLMs were undertrained.
4. **The 20× rule:** Train on ∼20 tokens per parameter for compute-optimality.
5. **Beyond compute-optimal:** Inference cost pushes toward *smaller models trained longer*.
6. **Open frontiers:** Data quality, inference-time compute, multi-epoch training, MoE scaling.

For Next Time

Read: J&M Ch. 10 (fine-tuning and adaptation) / Hu et al. (2022) – LoRA.

References I

Primary readings for this lecture:

Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.

Additional references:

Brown, T.B., et al. (2020). Language models are few-shot learners. *NeurIPS 2020*.

Clark, A., et al. (2022). Unified scaling laws for routed language models. *arXiv:2202.01169*.

Muennighoff, N., et al. (2023). Scaling data-constrained language models. *arXiv:2305.16264*.

Rae, J., et al. (2021). Scaling language models: Methods, analysis & insights from training Gopher. *arXiv:2112.11446*.

Schaeffer, R., Miranda, B., & Koyejo, S. (2024). Are emergent abilities of large language models a mirage? *NeurIPS 2023*.

Sutton, R. (2019). The bitter lesson. *Blog post*.

Touvron, H., et al. (2023a). Llama: Open and efficient foundation language models. *arXiv:2302.13971*.

Touvron, H., et al. (2023b). Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*.

Wei, J., et al. (2022). Emergent abilities of large language models. *TMLR*.